Capture the Flag 6: Active or Inactive?

🏴 Flag 0 — Create Your Feature Branch

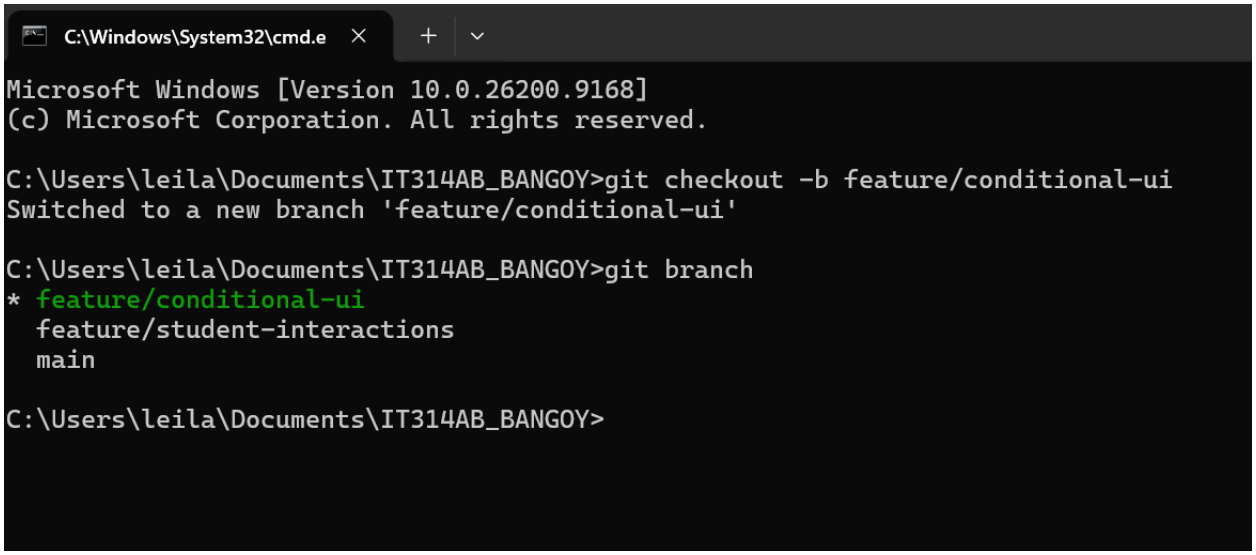
Objective: Create a new feature branch before adding conditional UI.

Create:

`feature/conditional-ui`

Screenshot:

- Terminal showing your current branch



🏴 Flag 1 — Show Active or Inactive

Objective: Change the Student Card based on whether a student is active or inactive.

Add an `active` status to your student data.

Research how Flutter uses **conditional expressions** to display different widgets.

Example states:

- Active
- Inactive

Test

Active students should display an Active indicator.

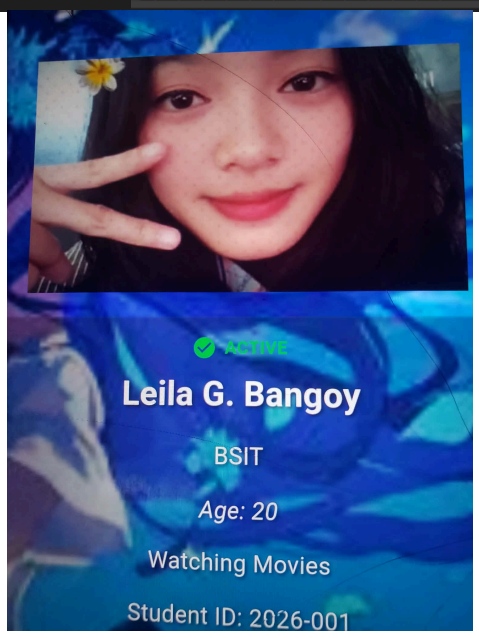
Inactive students should display an Inactive indicator.

Screenshot:

- Active Student Card
- Inactive Student Card
- Conditional logic in your code

```
    'yearLevel': 3,
    'age': 20,
    'hobby': 'Watching Movies',
    'studentId': '2026-001',
    'email': 'leila@gmail.com',
    'favoriteSubject': 'PE',
    'editing': false,
    'active': true,
  },
  {
    'image': 'assets/Profile2.jpg',
    'name': 'Karol S. Pal',
    'course': 'BSIT',
    'yearLevel': 3,
    'age': 20,
    'hobby': 'Sleeping',
    'studentId': '2026-002',
    'email': 'karol@gmail.com',
    'favoriteSubject': 'Theo',
    'editing': false,
    'active': false,
  },
```

```
            ? Colors.green
            : Colors.red,
        size: 20,
      ), // Icon
      const SizedBox(width: 6),
      Text(
        student['active']
            ? 'ACTIVE'
            : 'INACTIVE',
        style: TextStyle(
          fontSize: 14,
          fontWeight: FontWeight.bold,
          color: student['active']
              ? Colors.green
              : Colors.red,
        ), // TextStyle
      ), // Text
    ],
  ), // Row
  const SizedBox(height: 8),
  Text(
    student['name'] ?? 'Unknown',
```



🏴 Flag 2 — Show or Hide

Objective: Display certain UI elements only when a condition is true.

Add a conditional element to the Student Card.

Examples:

- Favorite badge appears only when favorited
- Warning appears only when inactive
- Delete action appears only if active

Research ways to conditionally include or exclude widgets from the widget tree.

Test

When the condition is true, the element appears.

When the condition is false, the element disappears.

Screenshot:

- UI with the element visible
- UI with the element hidden
- Conditional widget code

🏴 Flag 3 — The Empty Directory

Objective: Handle situations where there are no students to display.

Modify your Student Directory so it detects when the student list is empty.

Create an **empty state**.

Examples:

- "No students found"
- Empty-state icon
- Button to add a student

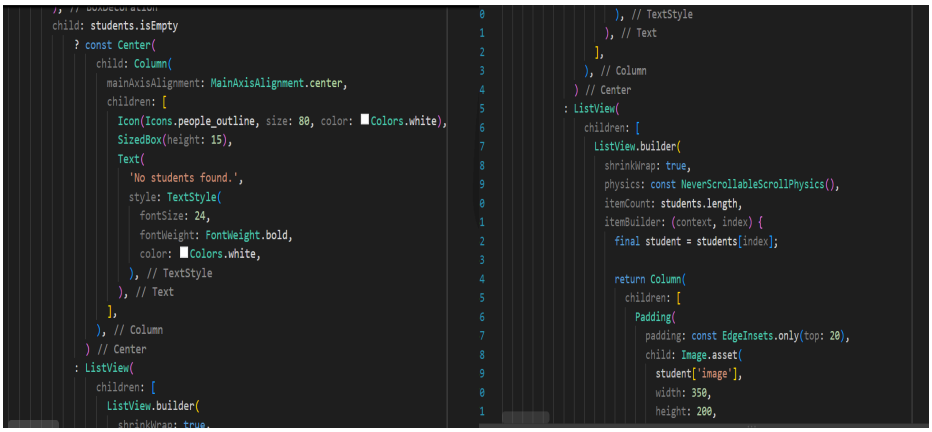
Test

Delete all students in your application (not the code)

Instead of showing a blank screen, your application should display your empty state.

Screenshot:

- Empty student list
- Empty-state UI
- Condition checking for an empty list



🏴 Flag 4 — Loading the Directory

Objective: Display different UI while student data is being loaded.

Create a loading state for your Student Directory.

Research how Flutter can conditionally display a loading indicator.

Examples:

- `CircularProgressIndicator`
- Loading message
- Loading placeholder

Test

When the application is in a loading state, the student list should not be displayed as the main content.

Once loading is complete, the Student Directory should appear.

Screenshot:

- Loading state
- Loaded state
- Conditional loading logic

```
child: isLoading
    ? const Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            CircularProgressIndicator(),
            SizedBox(height: 15),
            Text(
              'Loading students...',
              style: TextStyle(
                fontSize: 20,
                fontWeight: FontWeight.bold,
                color: Colors.white,
              ), // TextStyle
            ), // Text
          ],
        ), // Column
      ) // Center
    : students.isEmpty
    ? const Center(
        child: Text(
```

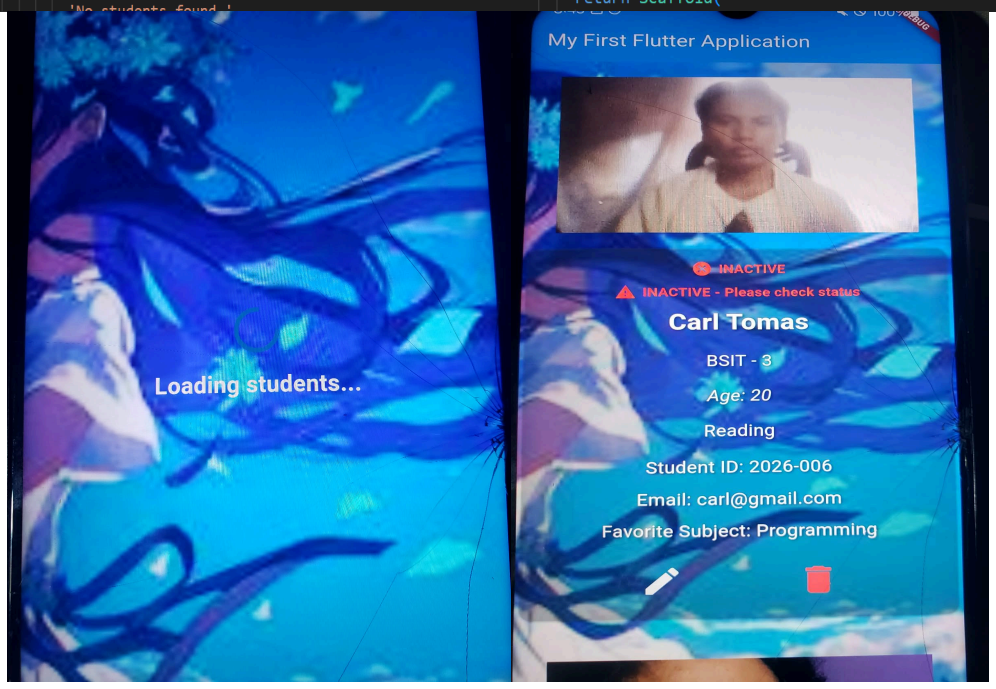
```
  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  bool isLoading = true;

  @override
  void initState() {
    super.initState();

    Future.delayed(const Duration(seconds: 2), () {
      setState(() {
        isLoading = false;
      });
    }); // Future.delayed
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
```



🏴 Flag 5 — Multiple States

Objective: Make your Student Directory handle multiple UI states correctly.

Your application should now be able to display:

1. Loading
2. Students available
3. No students available

Determine which state should be displayed based on your data.

Test

Verify each state:

Loading
→ Loading UI

Students available
→ Student List

Empty list
→ Empty State

Only the appropriate UI should appear at a time.

Screenshot:

- Loading state
- Student list state
- Empty state

🏴 Flag 6 — States Work Together

Objective: Make sure conditional UI does not break your existing interactions.

Test the combination of:

- Favorite
- Delete
- Edit
- Active/Inactive
- Show/Hide
- Empty state
- Loading state

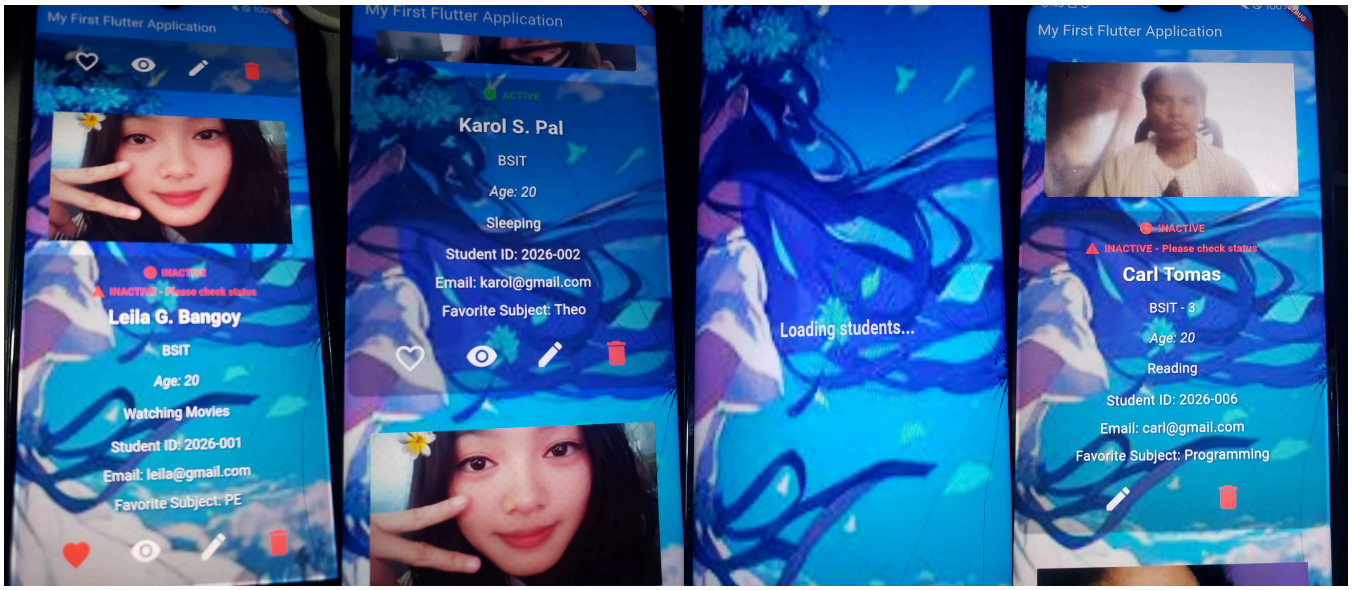
Test Scenario

1. Favorite a student.
2. Change or display their Active/Inactive state.
3. Delete students until the directory becomes empty.
4. Verify the Empty State appears.
5. Reload or restore students.
6. Verify the Student Directory appears again.

The application should continue functioning correctly as its state changes.

Screenshot:

- Multiple conditional states working
- Interactive Student Card
- Empty State
- Application functioning normally



🏴 Flag 7 — Save Your Progress

Objective: Commit and push your completed Conditional UI Student Directory.

Use a proper Git commit message.

Branch:

`feature/conditional-ui`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit